



Fast but Fragile

Navigating the Pitfalls of Rapid Development

Eric Greene | Xebia



Table of contents

Why AI Shifts Quality

Volume, plausibility, and the "looks right" problem

Quality Dimensions

Correctness, maintainability, security, and tests

Metrics That Matter

Signals worth tracking and vanity metrics to avoid

Where AI Code Breaks

Logic errors, outdated patterns, hallucinations, security gaps

Building a Safety Net

Human review, automated gates, static analysis, CI

Standards First

Instructions and specs that steer AI toward quality

Why AI Changes the Quality Conversation

Volume

More code than reviewers
can read closely

Plausibility

Clean names, docstrings,
and confident structure

“Looks right”

Reviewers relax when
code reads well

The bottleneck moves from writing code to verifying it.

Where AI Code Tends to Break

1

Subtle logic errors

```
if amount > 100:
```

The requirement said "\$100 or more"

2

Outdated patterns

```
datetime.utcnow()
```

Deprecated since Python 3.12

3

Hallucinated APIs

```
statistics.quantile(totals, 0.95)
```

A plausible name that doesn't exist

4

Silent security gaps

```
f"... WHERE email = '{email}'"
```

Works perfectly for honest input



Demo 1 · about 15 minutes

The “Looks Right” Problem

`demos/01-looks-right`

Quality Dimensions That Matter Most

1

Correctness

Does what the requirement says, at every boundary

AI risk

Matches the request's shape, misses its details

2

Maintainability

Readable, typed, and consistent with the codebase

AI risk

Mixed styles and outdated idioms

3

Security

Safe with hostile input and unguessable secrets

AI risk

Insecure patterns copied from training data

4

Test coverage

Tests that fail when the code is wrong

AI risk

Tests written to pass, not to catch bugs

Metrics Worth Tracking, and Metrics That Mislead

Worth tracking

Mutation score

Would the tests catch a bug?

Escaped defects

What got past review and testing?

Change fail and rework rates

DORA's measures of unstable delivery

Misleading on their own

Lines of AI-generated code

Volume isn't value

Suggestion acceptance rate

Measures trust, not correctness

Coverage percentage alone

Proves lines ran, nothing more

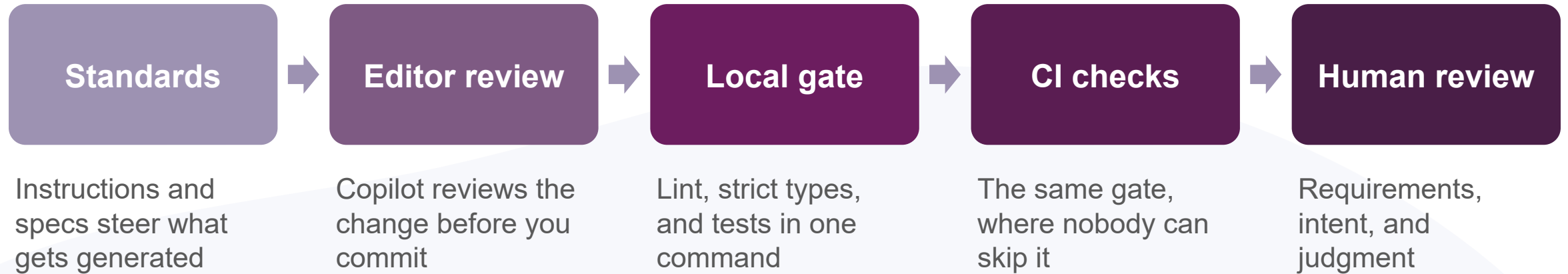


Demo 2 · about 15 minutes

Coverage Isn't Confidence

`demos/02-coverage-isnt-confidence`

Building a Layered Safety Net



Automate what a tool can check, so reviewers spend attention where only people can help.

Gates Tuned for AI Output

- Type checking catches hallucinated APIs
- Security rules flag injection and weak randomness
- Deprecation warnings fail the test run
- Coverage is a floor, not a goal
- Fix findings; never suppress them

```
pyproject.toml

[tool.ruff.lint]
select = ["E", "F", "B", "UP",
          "S", "DTZ", "BLE"]

[tool.mypy]
strict = true

[tool.pytest.ini_options]
filterwarnings = [
    "error::DeprecationWarning"]
```



Demo 3 · about 20 minutes

Automated Gates and the Copilot Review Loop

[demos/03-safety-net](#)

Team Standards Copilot Actually Reads

`.github/copilot-instructions.md`

Repo-wide standards for every chat, agent, and review

`.github/instructions/*.instructions.md`

Rules scoped to file patterns with applyTo

`.github/prompts/*.prompt.md`

Reusable team prompts, run as slash commands

Start with the comments you keep leaving in code review.

Spec-Driven Development with Spec Kit



```
specify init --here --integration copilot
```

Adds /speckit-* commands to Copilot Chat



Demo 4 · about 20 minutes

Standards First with Spec Kit and Copilot

`demos/04-standards-first`

Next Steps

Five changes you can start this month

Turn on strict typing and security lint rules

Fail the build on deprecation warnings

Report mutation score alongside coverage

Turn review comments into Copilot instructions

Pilot Spec Kit on one well-bounded feature





Questions?

Eric Greene

eric.greene@xebia.com

xebia.com

